【AI Coding】借助cursor实现业务需求全栈交付实践

原创 用户互动团队 大淘宝技术 2026年2月6日 01:24 浙江 246人

本文记录了借助Cursor完成红包签到页“单品返子玩法”前端功能全栈交付的实践过程，涵盖需求拆解、AI生成方案与代码、弹窗动画（Lottie衔接+飞入效果）、Feeds插卡与去重、问题排查（预加载失效、回退刷新、状态管理等），强调AI辅助下需人工把控逻辑、样式和交互细节，并总结了日志定位、知识补缺等关键经验。

01 背景

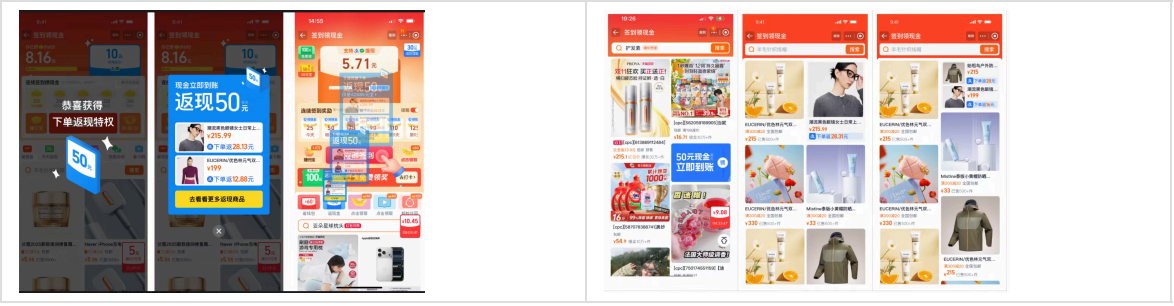
伴随着AI模型和工具能力的快速发展，AI coding正在不断的改变着我们日常的研发模式，同时也拓展了我们的技术边界。借助AI工具，可以快速的去了解一个代码库的业务逻辑和使用的技术栈，甚至基于此做迭代研发。本文主要从一个服务端的视角出发，使用AI工具（cursor）完成一次实际业务需求AI coding 的全栈交付实践，主要包含前端的交互逻辑、样式、动画等内容，服务端部分忽略。 主要目的是对AI coding过程中遇到的问题记录、总结和尝试，当然目前对前端的知识了解还比较浅薄，过程中如果有理解和使用上的偏差欢迎指正～

02 需求简介

核心功能：在红包签到首页对单品返子玩法作引导强化。

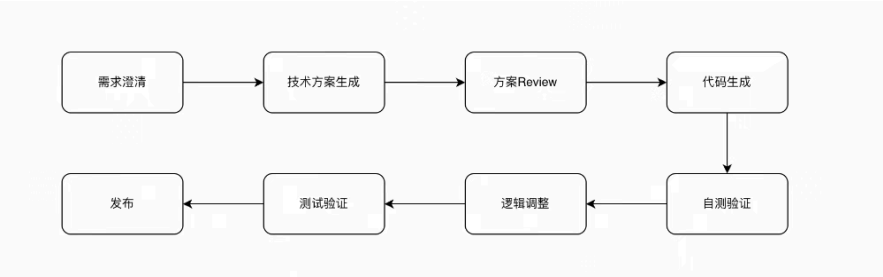
- 首页展示单品返子玩法商品POP，包括：弹窗前置动画、动画与POP过度、关闭弹窗收入二排动画等；
- 首页Feeds展示单品返子玩法商品卡片，包括：单商品卡片和多商品卡片两种样式。

单品返首页弹窗	单品返首页插卡



03 AI Coding实践

AI coding，关键流程和日常研发流程类似，只是更多的是让AI去做具体的执行，而我们做好过程管理、决策和CR部分。一般我们会遵循以下流程：



下面从这个具体的需求出发，展开实践过程。

STEP 1: 确定需求点、修改范围，编写需求描述

- 需求澄清

首页根据PRD和设计稿的要求，进行需求澄清，提炼需求点。再结合对工程了解程度，确定大致的方案和修改点，如果对需要修改的工程结构比较陌生，也可以借助cursor帮忙总结。

初步分析本次需求，前端涉及的改动：

- 首页新增一个Pop（需要回调疲劳度、动画、Pop上需要展示商品）–只在老版本；
- 首页Feeds增加单品返商品插卡（可复用首页渠道插卡） – 新老版本都需要；
- 首页Feeds和单品返插卡商品双向去重 – 新老版本都需要。

涉及的前端工程：hbqd5（老版本主页）、qd（新版本主页）、order–withdraw–feeds（子玩法）。

- 创建变更

分仓库建迭代、拉变更（不同业务的前端发布平台可能不同，具体可咨询对应前端同学），编写需求描述– task.md

STEP 2: 使用AI工具编写实现方案，并review方案

- AI初版方案

使用Cursor的plan模式，引用编写好的需求澄清文档，生成执行计划，并review，持续和AI交互迭代

(待确认的问题，这里忘记截图了，，，，

cursor会根据编写的执行计划中不确定的点给出待确认的问题，问题一般预设好了答案/是在不确定的会让你自己给答案，对预设答案的问题，还会标识推荐选项。按序号回复即可。

- 方案Review

对编写的执行计划进行review，有任何觉得不可行的实现都可以进行修改（这里还没输出到solution.md....

生成执行计划之后，若还有其他问题，可以继续确认再修改执行计划，直到方案满足要求

STEP 3: AI 生码，完成需求基础框架

前两步已完成执行方案生成，这里可以按照方案，让AI工具一次性完成前端需求的大体框架和内容

已完成的工作

1. 单品返Feeds接口服务

- 创建了 `querySingleItemCashBackFeeds.ts`，支持 `singleItemExcludeItemIds` 参数

2. 常量定义

- 在 `BACKEND_POP_TYPE` 和 `MODAL_TYPE` 中添加了 `SINGLE_ITEM_CASH_BACK_ITEMS`

3. POP展示逻辑

- 在 `showPopList.ts` 中添加了新的POP类型到主数据POP列表
- 在 `Modal/index.tsx` 的 `REPEAT_MODAL` 中添加了前端会话去重
- 在 `SignIn54/index.tsx` 中实现了POP展示逻辑（调用接口、取前两个商品、传递数据）

4. 弹窗组件

- 创建了 `SingleItemCashBackItems` 弹窗组件
  - 展示前置动画（ModalBgAnim）
  - 展示两个商品卡片
  - 实现了飞入动画（从商品卡片到二排单品返坑位）
  - 实现了疲劳度回调
- 在 `Modal/config.tsx` 中注册了新弹窗配置

5. 渠道卡片插卡

- 在 `Feeds/index.tsx` 中实现了时序控制逻辑
  - 追踪第一页和第二页的加载状态（支持SSR和非SSR场景）
  - 识别 `single_item_card_1` 和 `single_item_card_2` 类型
  - 从第三页开始传递 `singleItemExcludeItemIds` 参数
- 创建了 `SingleItemChannelCard` 组件
  - 等待第一页和第二页加载完成后再请求单品返Feeds
  - 根据 `bizType` 展示对应数量的商品
  - 处理空结果（不展示卡片）
- 在 `api.ts` 中添加了 `singleItemExcludeItemIds` 参数支持

所有代码已实现并通过基本检查。可以开始测试了。

STEP 4: 自测并完善样式、动画、交互等功能

- 弹窗样式细节调整

本地调试观察实现和交互稿的差异，直接通过视觉稿截图和AI交互，让其修改(这里可以使用D2C插件，生成样式，但当时比较头铁....

经验手调：结合视觉交互稿中的布局、字体、尺寸，调整样式文件

- 动效实现

动效和视觉确认后，拆分成两个部分，Pop前使用lottie动画播放，Pop后入口引导手搓一下。

1. lottie动画实现

```
<LottieAnim
  option={{
    url: lottieUrl,
  }}
  containerClassName={styles.lottieWrapper}
  onEnd={handleLottieEnd}
  onLoad={handleLottieLoad}
  play={[{ name: 'clip1', playCount: 1 }]}
/>
```

```
export interface LottieAnimProps {
  option: {
    // 资源地址
    url: string;
    // 降级图片地址
    downgradeUrl?: string;
  };
  containerClassName?: string;
  containerStyle?: any;
  // 动画结束回调
  onEnd?: () => void;
  // 加载完成
  onLoad?: () => void;
  // 降级后多少秒后onEnd
  downgradeDelay?: number;
  // 播放配置
  play?: { name: string; playCount: number }[];
}
```

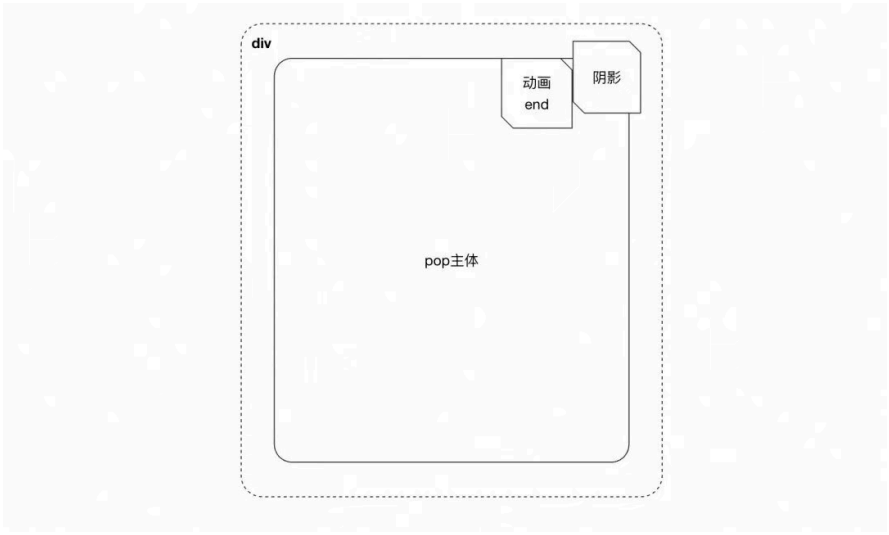
lottie动画简单来说，就是播一段素材就行，代码库中有现成的组件，比较难的点是，动画和静态素材之间的过渡。需要但前置动画播放完之后，正好衔接POP弹窗，动画的结束位置在POP弹窗的顶部。

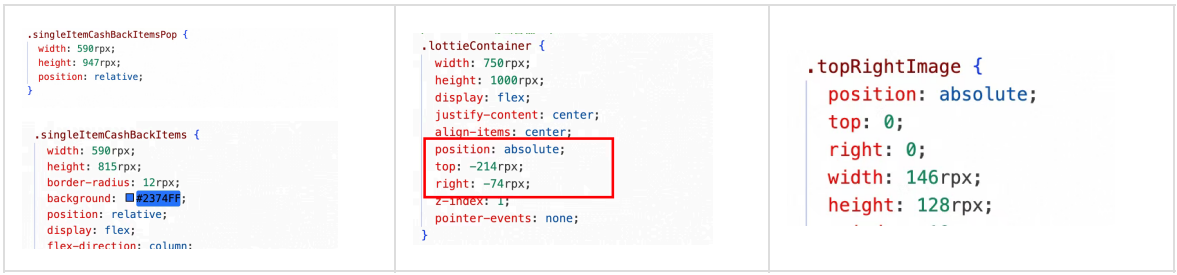
那这里就需要拆成两步，基础实现+微调。基础的功能实现交给AI：

pop弹窗和lottie动画的衔接处理：增加弹窗状态，通过lottie动画组件回调控制弹窗的隐藏和可见。

在onLoad中设置sleep，大致在动画播放完成时正好让弹窗可见，达到视觉效果。

动画结束位置、阴影表达：通过布局样式定位。





```
.singleItemCashBackItemsPop {
  width: 590rpx;
  height: 947rpx;
  position: relative;
}

.singleItemCashBackItems {
  width: 590rpx;
  height: 815rpx;
  border-radius: 12rpx;
  background: #2374FF;
  position: relative;
  display: flex;
  flex-direction: column;
```

```
.lottieContainer {
  width: 750rpx;
  height: 1000rpx;
  display: flex;
  justify-content: center;
  align-items: center;
  position: absolute;
  top: -214rpx;
  right: -74rpx;
  z-index: 1;
  pointer-events: none;
}
```

```
.topRightImage {
  position: absolute;
  top: 0;
  right: 0;
  width: 146rpx;
  height: 128rpx;
```

最外层样式itemsPop设置position：relative；动画和pop主体平级，动画样式设置position：absolute。

使用relative为子组件提供一个“锚点”

1. **作为绝对定位元素的容器（最常见）**：当你需要在一个元素内部精确放置另一个元素时（如图标、徽章、模态框、下拉菜单等）。
2. **进行细微的位置调整**：当你需要稍微移动一个元素，但又不想破坏周围的布局时。例如，让一个元素与另一个轻微重叠。
3. **为了使用 z-index**：一个元素必须拥有 position 属性（非 static）才能让 z-index 生效，从而控制元素的堆叠顺序。position: relative; 可以在不改变布局的情况下激活 z-index。

全屏 复制

特性	position: static (默认)	position: relative	position: absolute
是否在文档流中	是	是 (原来的空间被保留)	否 (完全脱离，不占空间)
top/right/etc 的作用	无效	相对于自身原始位置进行移动	相对于最近的已定位祖先进行移动
能否成为定位容器	否	能	能

记住这个黄金组合：父相（relative）子绝（absolute），你就掌握了 position: relative; 的精髓。

通过过程中和AI的不断交互，补充知识盲区，最终实现效果。

2. 飞入动画实现

原始诉求：点击或者关闭时，Pop需要缩放收入二排的单品返坑位。

单品返现商品弹窗用户点击关闭按钮或者点击弹出内容跳转单品返页面前，有个动画效果：弹窗收到二排中单品返现卡片（type: singleItemCashBack）位置

修改内容：

1. 修改 `flyClose` 函数：
   - 起始位置：整个弹窗容器（`NODE_ID.SINGLE_ITEM_CASH_BACK_MAIN_ID`）
   - 目标位置：二排中的单品返现卡片（`BUSINESS_TYPE.singleItemCashBack`，即 `id="singleItemCashBack"`）
   - 使用 `hideNode` 参数：动画开始时隐藏弹窗
   - 在 `callBackEnd` 中先调用 `closeModal()`，再执行回调
2. 修改点击事件处理：
   - 移除了所有在 `flyClose` 之前调用 `closeModal()` 的代码
   - 点击商品卡片、主按钮、关闭按钮时，先执行动画，动画完成后再关闭弹窗

动画流程：

1. 用户点击（关闭按钮/商品卡片/主按钮）
2. 触发动画：弹窗从当前位置飞向单品返现卡片位置
3. 动画过程中：弹窗逐渐隐藏（`hideNode` 设置 opacity）
4. 动画完成后：调用 `closeModal()` 关闭弹窗，执行回调（如果需要跳转则跳转）

现在点击关闭按钮或点击弹窗内容跳转前，都会先播放收起动画，弹窗收起到单品返现卡片位置。

具体实现：计算飞入动画的开始和结束位置，调用飞入动画函数，给定动画时间，复用已有的飞入动画组件。

```
// 先获取元素引用，确保在关闭弹窗前元素存在且可见
const startTarget = document?.getElementById(NODE_ID.SINGLE_ITEM_CASH_BACK_MAIN_ID);
const endTarget = document?.getElementById(BUSINESS_TYPE.singleItemCashBack);

// 先关闭弹窗
if (shouldCloseModal) {
  closeModal();
}

// 如果找不到元素，直接执行回调（不执行动画）
if (!startTarget || !endTarget) {
  closeCallback && closeCallback();
  return;
}

// 同步获取元素引用后，立即执行 fly 动画
// fly 函数会克隆元素，所以即使原元素被移除，克隆元素也会继续动画
fly({
  start: startTarget,
  end: endTarget,
  options: {
    duration: 0.6,
  },
  callBackEnd: () => {
    // 动画完成后执行回调
    closeCallback && closeCallback();
  },
});
```

AI实现过程遇到的问题（谁写的Bug谁修 😅 ...AI修）：

- 先关闭了弹窗，然后再获取起始位置，导致无法获取开始位置，动画不生效

测试了一下，弹窗并不会飞向二排，你再仔细思考下

重新分析：`closeModal()` 会立即触发 React 状态更新，可能导致 DOM 元素被移除。检查 `fly` 函数实现，并确保在关闭弹窗前获取元素：

Explored 3 files 2 searches

`RootModal` 的 `visible` 属性会在 `closeModal()` 后立即隐藏或移除 DOM。需要在关闭前先获取元素和位置信息。修复：

index.tsx +35 -21

```
    // 先获取元素引用，确保在关闭弹窗前元素存在
    // 先获取元素引用和位置信息，确保在关闭弹窗前元素存在且可见
    const startTarget = document?.getElementById(NODE_ID.SINGLE_ITEM_CASH_BACK_MAIN_ID);
```

1 个隐藏的行

- 动画重复，一个弹窗缩放的，一个飞入二排的

现在的动画过渡效果有点冗余，只需要保留飞向二排位置的即可

简化动画，仅保留 fly 函数实现的飞向二排位置的动画，移除弹窗本身的 CSS 过渡：

Read index.tsx L18-82

index.tsx +2 -36

```
    // 同时使用fly函数创建克隆元素的动画，提供更好的视觉反馈
    // 使用fly函数创建飞行动画：弹窗飞向二排位置
    fly({
      end: endTarget,
```

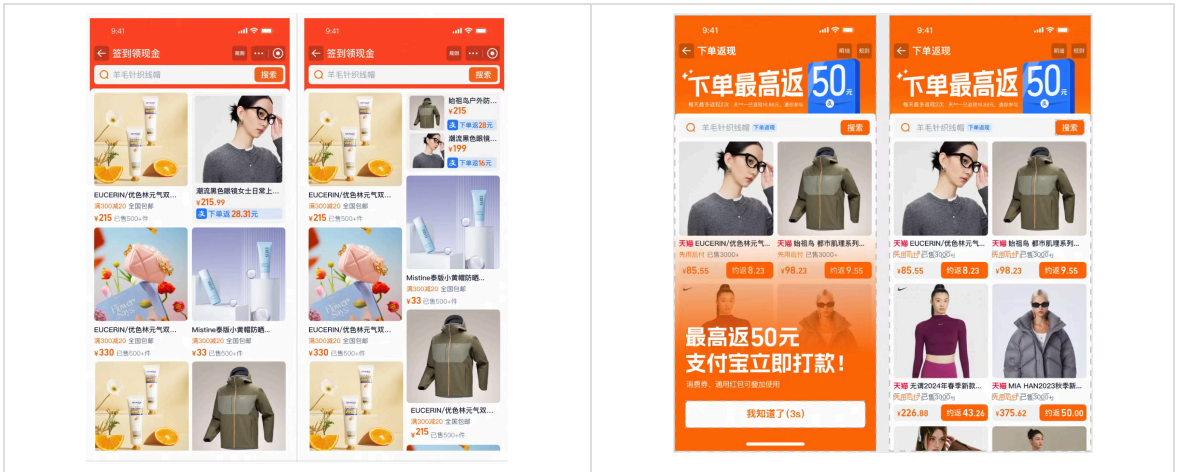
1 个隐藏的行

- Pop动画和商品加载完成后，点击Pop无跳转效果
  - 怀疑是动画添加后，覆盖了什么组件，导致点击事件失效；
  - 让cursor增加点击事件日志，根据控制台输出的日志，排查问题；
  - 调整lottie动画的z-index：1，Pop其他组件z-index调高，避免被覆盖。

- Feeds商品插卡-签到首页Feeds和渠道卡片商品去重

Feeds去重服务端处理，但需要前端请求Feeds接口时带上已曝光的商品列表

- 签到首页：携带单品返现渠道卡片曝光商品
- 单品返渠道卡片：携带首页Feeds曝光商品



实现过程中遇到的问题与优化：

1. 预期的预加载的逻辑和实际不符

签到首页Feeds有预加载逻辑，在首次进入页面或者下拉刷新时，请求Feeds第一页时，会预加载第二页请求，通过Feeds插件配置实现，但实际表现是：当用户在首页加载完成后，滑动页面，加载完第二页后，才会加载单品返现插卡商品。

写让cursor确认下逻辑

重新review下，首页Feeds和单品返渠道卡片Feeds请求时序和去重逻辑

Read querySingleItemCashBackFeeds.ts

Listed feeds

Grepped querySingleItemCashBackFeeds|首页Feeds|单品返

Read index.tsx

Thought for 1s

查看相关组件和接口实现：

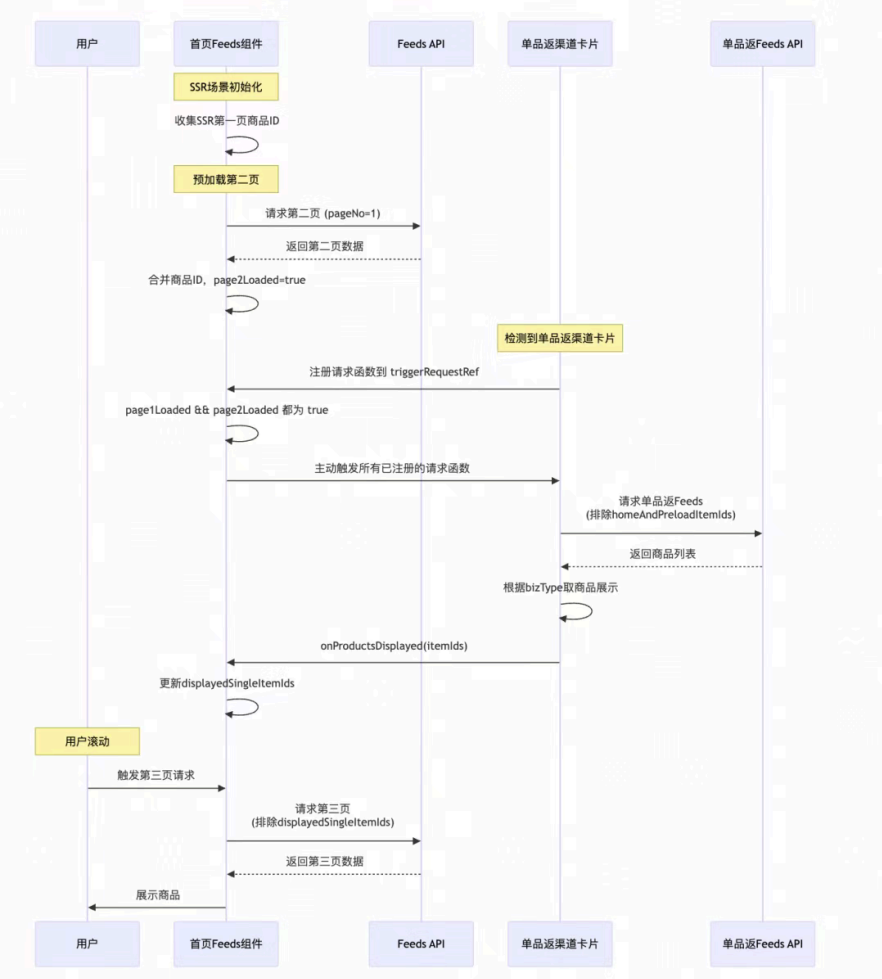
好的，再整体复盘下逻辑，输出一个流程图，让我确认下

Thought for 1s

梳理完整逻辑，绘制流程图。先查看相关代码：

Read index.tsx L50-74

```
1. 初始化 (SSR/CSR)
   ↓
2. 第一页加载完成 (CSR) / SSR第一页已存在
   ↓
3. 第二页预加载完成
   ↓
4. 单品返渠道卡片请求 (使用 homeAndPreloadItemIds 排除)
   ↓
5. 渠道卡片展示商品，更新 displayedSingleItemIds
   ↓
6. 第三页及以后请求 (使用 displayedSingleItemIds 排除)
```



看起来链路上一切都没问题，那就继续找吧

流程图-首页Feeds和单品返渠道卡... 按照这里的逻辑流程图描述，应该不需要用户滑动，就回加载单品返渠道卡片，但实际表现是，只有向下滚动才会出发点加载渠道卡片，请梳理逻辑，查找原因，并给出修复方案

Thought for 2s

分析问题：检查单品返渠道卡片的渲染和触发逻辑：

Read index.tsx L39-88

说是状态判断的问题：

问题分析

根本原因：

1. `SingleItemChannelCard` 的 `useEffect` 只在组件创建时执行
2. `renderCard` 只在卡片需要渲染时调用（虚拟滚动/懒加载）
3. 若渠道卡片不在首屏，组件不会被创建，`useEffect` 不会执行
4. 只有滚动到卡片位置时才会触发请求

修复方案

采用"注册-触发"机制：

1. 在父组件中添加 `triggerSingleItemRequestRef`，用于存储所有单品返渠道卡片的请求函数
2. 当 `page1Loaded && page2Loaded` 都为 true 时，主动触发所有已注册的请求
3. 在 `SingleItemChannelCard` 组件中，将请求函数注册到父组件的 ref 中

修复了之后依旧有问题，继续找

实际测试下来，在CSS页面，会在pageNum为2请求之后，才会请求单品返Feeds，并且会请求两次单品返Feeds，后再渲染单品返渠道卡片，这不太符合预期，你再仔细想想思考思考

Thought for 3s

分析问题：检查代码逻辑，找出重复请求和时序问题的原因。

Read index.tsx L71-80

问题分析

1. 重复请求的原因：
   - 父组件的 useEffect 在 `page1Loaded && page2Loaded` 时触发
   - 子组件的 useEffect（兜底机制）也在 `page1Loaded && page2Loaded` 时触发
   - 两个 useEffect 都会触发请求，导致重复
2. 时序问题：
   - CSR 场景下，`pageNo=2` 请求完成后才设置 `page2Loaded=true`
   - 如果渠道卡片不在首屏，此时组件还未挂载，未注册请求函数
   - 父组件的 useEffect 已执行，但当时没有注册的函数
   - 用户滚动到卡片位置时，组件才挂载并注册，但父组件的 useEffect 不会再次执行

修复方案

1. 移除子组件的兜底 useEffect，只保留注册机制
2. 在子组件注册时，如果 `page1Loaded && page2Loaded` 已为 true，立即触发请求
3. 父组件的 useEffect 主要用于 SSR 场景
4. CSR 场景在 `pageNo=2` 完成时直接触发请求

重复请求的问题fix了，没解决滑动加载的问题

重复的问题解决了，但还是只会在主页pageNum=2请求之后，才会加载单品返渠道卡片，你再仔细想想

Thought for 8s

分析问题：检查当前逻辑，找出为什么只在 pageNum=2 之后才加载。

Read index.tsx L449-493

Read index.tsx L92-116

Searched When does FeedsList call renderCard? Is it called immediately when data is loaded or only when scrolling into view?

修复方案

1. 统一触发机制

- 移除 SSR 场景的特殊判断，统一在 `page1Loaded && page2Loaded` 时触发
- 这样可以处理组件在第二页加载完成后才挂载的情况

2. 双重保障

- 在 `pageNo=2` 完成时直接触发（CSR 场景）
- 通过 useEffect 监听 `page1Loaded && page2Loaded` 的变化，处理组件延迟挂载的情况

3. 子组件注册时的检查

- 如果注册时 `page1Loaded && page2Loaded` 已经都是 true，立即触发请求
- 使用 setTimeout 确保在下一个事件循环中执行，避免时序问题

修复后的流程

CSR场景：

1. `pageNo=1` 完成 → `page1Loaded=true`，不触发请求（排除参数不完整）
2. `pageNo=2` 完成 → `page2Loaded=true`，立即触发所有已注册的请求
3. 如果组件在第二页完成后才挂载，注册时检查到条件已满足，立即触发

SSR场景：

1. `pageNo=1`（对应第二页）完成 → `page2Loaded=true`，立即触发所有已注册的请求
2. useEffect 也会触发，但因为有 `hasRequested` 保护，不会重复请求

现在应该可以解决时序问题。请测试验证。

给出了修复方案，但依旧未解决，甚至罢工了。。。。

AI解决不了，那就“开发”看看...

看组件的实现逻辑，FeedsList -> hudong-common-feeds中的InfiniteList -> coinhub-react-infinite-list中waterfallList

```
<FeedsList
  listRef={handleListRef}
  bizCode={'hbqd'}
  sceneCode={'majorFeeds'}
  initData={ssrFeedsData}
  columnCount={count}
  startPageNo={ssrFeedsData?.data?.length > 0 ? 2 : 1}
  requestData={({ pageNo }) => {…
  }}
  renderCard={(data, index, cardObj) => {…
  }}
  plugins={{
    preload: new PreloadPlugin({
      firstScreenPreloadPages: 1,
    }),
  }}
/>
```

hudong-common-feeds:

```
return (
  <InfiniteList
    dataSpm={`hudongfeeds_${spmC}`}
    ref={listRef ?? innerListRef}
    initListData={initData?.data ?? []}
    skeletonDataStr={columnInfo.skeletonView}
    columnCount={columnCount ?? columnInfo.count}
    requestData={async (requestParams: any) => {…
    }}
    renderItem={(item: any, id: any, cardInfo: any) => {…
    }}
    {...restListProps}
    plugins={{
      innerTracker: new innerTrackerPlugin({
        bizCode: bizCode,
        sceneCode: sceneCode,
        spmC: `hudongfeeds_${spmC}`,
        ...restListProps,
      }),
      ...(plugins || {}),
    }}
  />
);
};
```

coinhub-react-infinite-list包：

```
function WaterfallList(props: IProps, ref) {
  const {
    initListData,
    requestData,
    skeletonDataStr,
    skeletonView,
    startPageNo = 0,
    tabIndex,
    columnCount,
    skeletonSize = 10,
    renderItem,
    tabInfo = {},
    plugins = {},
    dataSpm = '',
    tabFeedsSuffix = '',
    style = {},
    openSkeleton = true,
    customNoMoreView,
    ...restProps
  } = props;
```

找到Feeds初始化逻辑

让cursor帮忙确认这里逻辑：

帮我确认一个问题，当我在外部调用这个WaterfallList组件，同时在plugin中设置了firstScreenPreloadPages==1，会出现预加载不成功的情况嘛？如果会出现，请分析原因，不要修改代码

分析 `firstScreenPreloadPages==1` 时预加载可能失败的情况。查看相关代码逻辑：

Explored 2 files

查看 `requestParamsRef` 的初始状态和 `initListData` 的处理逻辑：

Explored 1 file

分析 `firstScreenPreloadPages==1` 时预加载可能失败的情况：

场景分析

场景1：单个列表（`mainTabIndex === undefined`）+ 没有 `initListData` + `firstScreenPreloadPages == 1`

执行流程：

```
T0: useEffect 开始执行
    ├─ requestParamsRef.current.isInit = false (因为没有 initListData)
    └─ requestParamsRef.current.isRequesting = false

T1: 第322行检查条件
    ├─ cacheListRef.current.mainTabIndex === undefined ✓
    └─ !requestParamsRef.current?.isInit = true ✓
    └─ 条件满足，进入 if 块

T2: 第323行执行 getMoreList()
    ├─ 同步执行:
    │   ├─ 检查: hasMore && !isRequesting → true && true = true ✓
    │   ├─ 设置: isRequesting = true
    │   └─ 调用: presetSkeletonData(pageNo) (如果 openSkeleton && canAppend)
    └─ 异步执行:
        └─ requestAnimationFrame(() => { requestData(...) }) 排队等待

T3: 第325行执行 onNoTabListInited (同步)

T4: 第329行执行 onListInited (同步)
    └─ 插件内部逻辑:
        ├─ 检查: firstScreenPreloadPages === 0 → false，继续执行
        ├─ 检查: tabIndex === undefined → true
        └─ 调用: getMoreList(false)
            ├─ 检查: hasMore && !isRequesting → true && false = false ✗
            └─ 被阻止！不会发起请求
```

结论：预加载失败。原因：第323行已设置 `isRequesting = true`，插件的调用被第185行的检查阻止。

场景2：单个列表（`mainTabIndex === undefined`）+ 有 `initListData` + `firstScreenPreloadPages == 1`

执行流程：

```
T0: useEffect 开始执行
    ├─ requestParamsRef.current.isInit = true (因为有 initListData)
    └─ requestParamsRef.current.isRequesting = false

T1: 第322行检查条件
    ├─ cacheListRef.current.mainTabIndex === undefined ✓
    └─ !requestParamsRef.current?.isInit = false ✗
    └─ 条件不满足，跳过 if 块

T2: 第329行执行 onListInited (同步)
    └─ 插件内部逻辑:
        ├─ 检查: firstScreenPreloadPages === 0 → false，继续执行
        ├─ 检查: tabIndex === undefined → true
        └─ 调用: getMoreList(false)
            ├─ 检查: hasMore && !isRequesting → true && true = true ✓
            ├─ 设置: isRequesting = true
            └─ 发起请求: requestAnimationFrame(() => { requestData(...) })
```

结论：预加载成功。原因：第323行未执行，`isRequesting` 仍为 `false`，插件的调用可以正常执行。

场景3：`tabIndex === 0` + `firstScreenPreloadPages == 1`

执行流程：

```
T0: useEffect 开始执行
    ├─ cacheListRef.current.mainTabIndex = tabIndex (不为 undefined)
    └─ requestParamsRef.current.isRequesting = false

T1: 第322行检查条件
    ├─ cacheListRef.current.mainTabIndex === undefined → false ✗
    └─ 条件不满足，跳过 if 块

T2: 第329行执行 onListInited (同步)
    └─ 插件内部逻辑:
        ├─ 检查: firstScreenPreloadPages === 0 → false，继续执行
        ├─ 检查: tabIndex === 0 → true
        └─ 调用: getMoreList(false)
            ├─ 检查: hasMore && !isRequesting → true && true = true ✓
            ├─ 设置: isRequesting = true
            └─ 发起请求: requestAnimationFrame(() => { requestData(...) })
```

结论：预加载成功。原因：第323行未执行，插件的调用可以正常执行。

问题总结

会出现预加载不成功的情况。

失败条件：

- 单个列表（`mainTabIndex === undefined`）
- 没有传入 `initListData`（`isInit = false`）
- 插件配置了 `firstScreenPreloadPages == 1`

失败原因：

1. 执行顺序冲突：
   - 第323行先调用 `getMoreList()`，同步设置 `isRequesting = true`
   - 第329行插件再调用 `getMoreList(false)` 时，被第185行的 `!isRequesting` 检查阻止
2. 状态检查机制：

```
if (hasMore && !isRequesting) {  // 第185行
  requestParamsRef.current.isRequesting = true;
  // ...
}
```

由于 `isRequesting` 已在第323行被设置为 `true`，插件的调用无法通过检查

3. 设计问题：
   - 组件在第323行主动初始化
   - 插件在第329行也想初始化
   - 两者在同一 `useEffect` 中同步执行，产生冲突

影响

当满足失败条件时：

- 第323行的调用会正常执行（第一次请求）
- 插件的预加载调用会被阻止（第二次请求失败）
- `firstScreenPreloadPages == 1` 的预加载意图无法实现

到这里，大概明确了问题原因，CSR链路下，在预发环境下，受Feeds接口请求耗时的影响，预加载会失效，只有等待用户翻页，才会继续请求，SSR链路下，没问题。总结：组件的代码，首次请求和预加载的执行是同步的，但请求feeds接口是异步的，会存在预加载失败的问题，应该等待首次请求结束后，再执行预加载。

2. 首页Feeds直通车回退插卡，导致单品返插卡商品刷新

问题表现是：直通车回退时，商品卡片商品刷新。先和cursor交互，先定位问题，梳理逻辑，再解决问题。

最终优化方案

1. 使用全局 Map 记录已请求的组件

- 使用 `bizType + jumpUrl` 作为唯一标识符
- 即使组件因 key 变化而重新挂载，也能通过全局 Map 判断是否已请求

2. 稳定的 key

- 使用 `bizType + jumpUrl` 作为 key（不再依赖 `trueIndex`）
- `jumpUrl` 是稳定的，不会因列表更新而变化

3. 双重保护机制

- 全局 Map：`requestedComponentsMap` 记录已请求的组件
- 本地 loading 状态：防止同一组件实例的并发请求

核心代码逻辑

```
// 全局 Map，记录已请求的组件
const requestedComponentsMap = new Map<string, boolean>();

// 生成唯一标识符
function getComponentId(bizType: string, jumpUrl: string): string {
  return `single_item_card_${bizType}_${jumpUrl || 'default'}`;
}

// 请求前检查
if (requestedComponentsMap.get(currentComponentId) || loadingRef.current) {
  return; // 已请求过，直接返回
}

// 请求成功后标记
requestedComponentsMap.set(currentComponentId, true);
```

优势

1. 不依赖会变化的 key：即使 `trueIndex` 变化导致组件重新挂载，也能通过全局 Map 判断
2. 稳定的标识符：`bizType + jumpUrl` 组合在列表更新时不会变化
3. 请求失败可重试：失败时清除全局记录，允许重试

即使直通车回退导致 `trueIndex` 重新计算，组件重新挂载，也不会重复请求，因为全局 Map 已记录该组件已请求过。

请测试一下，应该可以解决问题。

感觉现在的实现逻辑很复杂，hasRequestedRef不能移动到父组件中去，但单品返卡片加载并接口请求后，记录下请求状态，这样，即使依赖列表改变了，在接口的请求逻辑中，也会因为已经请求过，不会继续请求

优化总结

1. 父组件管理请求状态

- 在父组件中添加 `requestedSingleItemCardsRef`，使用 `Set<string>` 记录已请求的组件
- 使用 `bizType + jumpUrl` 作为唯一标识符

2. 简化子组件逻辑

- 移除了全局 Map
- 移除了不必要的 ref（如 `loadingRef`、`componentId` ref 等）
- 直接使用 props 和简单的 ref 存储

3. 请求逻辑

- 请求前检查父组件的 `requestedCardsRef`
- 请求成功后，将标识符添加到父组件的 Set 中
- 请求失败时，从 Set 中删除，允许重试

核心代码

父组件：

```
// 单品返渠道卡片已请求记录
const requestedSingleItemCardsRef = useRef(new Set<string>());

// 传递给子组件
<SingleItemChannelCard
  requestedCardsRef={requestedSingleItemCardsRef}
  // ...其他 props
/>
```

子组件：

```
// 请求前检查
if (requestedCardsRef?.current?.has(componentId) || loading) {
  return;
}

// 请求成功后记录
requestedCardsRef.current.add(componentId);
```

优势

1. 逻辑更清晰：请求状态由父组件统一管理
2. 更稳定：即使依赖变化导致函数重新创建，也不会重复请求
3. 代码更简洁：移除了复杂的 ref 管理逻辑

即使直通车回退导致组件重新渲染或依赖变化，也不会重复请求，因为父组件的 `requestedSingleItemCardsRef` 已记录了请求状态。

这样改，确实不会重复请求了，但当直通车回退的请求返回后，原有的单品返渠道卡片就不展示了，请排查原因，给出解决方案，写不要改代码

问题原因分析

问题流程：

1. 初始状态：单品返渠道卡片请求成功，`products` 有数据，组件正常展示
2. 直通车回退：插入商品后，列表更新，`renderCard` 重新执行
3. 组件重新挂载：虽然 key 是 `single_item_card_${bizType}`，但如果列表结构变化导致组件位置改变，React 可能会重新挂载组件
4. 状态丢失：组件重新挂载时，`products` 状态被重置为空数组 `[]`
5. 不会再次请求：因为 `requestedCardsRef.current.has(cardId)` 返回 `true`，`requestSingleItemFeeds` 直接返回，不会再次请求
6. 结果：`products.length === 0`，`shouldRender` 为 `false`，组件不展示

核心问题：

- `requestedCardsRef` 只记录了"已请求"，没有缓存请求结果（products 数据）
- 组件重新挂载时，`products` 状态丢失，但不会再次请求来恢复数据

解决方案

方案1：在父组件中缓存 products 数据（推荐）

在父组件中不仅记录"已请求"，还要缓存请求结果：

```
// 父组件中
const requestedSingleItemCardsRef = useRef(new Set<string>());
const cachedProductsRef = useRef(new Map<string, any[]>()); // 新增：缓存 products 数据

// 传递给子组件
<SingleItemChannelCard
  requestedCardsRef={requestedSingleItemCardsRef}
  cachedProductsRef={cachedProductsRef} // 新增：传递缓存 ref
  // ...其他 props
/>
```

子组件逻辑：

1. 组件挂载时，先检查 `cachedProductsRef` 是否有缓存数据
2. 如果有缓存，直接使用缓存数据设置 `products`
3. 如果没有缓存，才发起请求
4. 请求成功后，将结果缓存到 `cachedProductsRef` 中

方案2：使用 useMemo 或外部状态管理

将 `products` 数据提升到父组件，使用状态管理库或 Context 来管理，这样即使组件重新挂载，数据也不会丢失。

方案3：调整 key 策略

确保 key 在列表更新时保持稳定，避免组件重新挂载。但当前 key 已经是 `single_item_card_${bizType}`，应该比较稳定了。

推荐使用方案1，因为：

- 改动最小
- 逻辑清晰
- 既能避免重复请求，又能保证数据不丢失
- 即使组件重新挂载，也能从缓存中恢复数据

需要我实现方案1吗？

请先梳理下这里数据的加载链路，从首页开始第一次渲染开始，到用户有点击商品，然后回退，可以搭配流程图，描述下单品返商品卡片数据的加载流转过程

梳理数据加载链路。先查看相关代码，理解完整流程：

Explored 2 files

继续查看关键代码：

Explored 2 files 1 search

梳理数据加载链路，并绘制流程图：

Read index.tsx L143-192

单品返商品卡片数据加载流转过程

流程图

Mermaid Syntax Error

View diagram source

详细流程说明

阶段1：首页首次渲染和初始化

1. 首页渲染
   - `FeedsList` 组件初始化
   - 如果有 SSR 数据，先展示并收集商品ID
2. 第1页数据加载

```
graph TD
    A[首页首次渲染] --> B[FeedsList 初始化]
    B --> C{是否有 SSR 数据?}
    C -->|是| D[展示 SSR 数据<br/>收集 SSR 商品ID到 homeExposedItemIdsRef]
    C -->|否| E[开始请求第1页数据]
    D --> E

    E --> F[fetchProducts pageNo=1]
    F --> G[收集第1页商品ID到 homeExposedItemIdsRef]
    G --> H[updatePageLoadedStatus 1<br/>firstTwoPagesLoaded = false]

    H --> I[自动请求第2页]
    I --> J[fetchProducts pageNo=2]
    J --> K[收集第2页商品ID到 homeExposedItemIdsRef]
    K --> L[updatePageLoadedStatus 2<br/>firstTwoPagesLoaded = true]

    L --> M[SingleItemChannelCard useEffect 触发]
    M --> N{检查 requestedCardsRef<br/>是否已请求?}
    N -->|否| O[调用 requestSingleItemFeeds]
    N -->|是| P[直接返回，不请求]

    O --> Q[设置 loading = true]
    Q --> R[将 cardId 添加到 requestedCardsRef]
    R --> S[请求 querySingleItemCashBackFeeds<br/>传入 homeExposedItemIdsRef 作为排除列表]

    S --> T{请求成功?}
    T -->|是| U[设置 products 状态<br/>调用 onProductsDisplayed 通知父组件]
    T -->|否| V[清除 requestedCardsRef 中的记录<br/>允许重试]

    U --> W[组件渲染展示商品]
    W --> X[用户点击商品]

    X --> Y[clickProduct 被调用]
    Y --> Z[跳转到详情页/单品返页面]
    Z --> AA[页面变为 hidden]
```

```
AA --> AB[用户点击返回]
AB --> AC[页面变为 visible]
AC --> AD[visibility.watch 触发<br/>updateFeedsVisible 'visible']

AD --> AE{检查 hitZtcAdBack<br/>是否为 'true'?}
AE -->|是| AF[获取 currentGoodsItemId 从 store]
AE -->|否| AG[结束流程]

AF --> AH[调用 queryMajorFeedsZtcBack<br/>获取直通车回退商品]
AH --> AI{直通车数据<br/>是否为空?}
AI -->|是| AG
AI -->|否| AJ[调用 internalListRef.updateListInfo]

AJ --> AK[在点击商品后面插入直通车商品]
AK --> AL[重新计算所有商品的 trueIndex<br/>addTrueIndexToList]

AL --> AM[列表更新, renderCard 重新执行]
AM --> AN{组件 key 是否变化?}
AN -->|是| AO[组件重新挂载<br/>products 状态丢失 = []]
AN -->|否| AP[组件重新渲染<br/>products 状态保留]

AO --> AQ{检查 requestedCardsRef<br/>是否已请求?}
AQ -->|是| AR[requestSingleItemFeeds 直接返回<br/>不会再次请求]
AQ -->|否| AS[重新请求]

AR --> AT[products.length === 0<br/>shouldRender = false]
AT --> AU[组件不展示 ❌]

AP --> AV[组件正常展示 ✅]
AS --> W
```

3. 第2页数据加载（自动预加载）

```
fetchProducts(pageNo=1)
↓
collectHomeExposedItemIds(1, products)
→ homeExposedItemIdsRef.current = [商品ID列表]
↓
updatePageLoadedStatus(1)
→ firstTwoPagesLoaded = false (还未完成)
```

阶段2：单品返渠道卡片请求

4. 触发单品返请求

```
fetchProducts(pageNo=2)
↓
collectHomeExposedItemIds(2, products)
→ homeExposedItemIdsRef.current = [第1页+第2页商品ID]
↓
updatePageLoadedStatus(2)
→ firstTwoPagesLoaded = true ✅
```

5. 请求逻辑

```
firstTwoPagesLoaded = true
↓
SingleItemChannelCard useEffect 触发
↓
requestSingleItemFeeds()
```

6. 请求成功处理

```
检查: requestedCardsRef.current.has(cardId)
→ 如果已存在, 直接返回 (防止重复请求)
→ 如果不存在, 继续执行

设置: loading = true
标记: requestedCardsRef.current.add(cardId)

请求: querySingleItemCashBackFeeds({
  singleItemExcludeItemIds: homeExposedItemIdsRef.current.join(',')
})
```

7. 组件渲染

```
设置: products = displayProducts
设置: loading = false
通知: onProductsDisplayed(displayedIds)
→ 父组件更新 displayedSingleItemIds
```

阶段3：用户点击商品

8. 点击处理

```
shouldRender = firstTwoPagesLoaded && !loading && products.length > 0
→ 如果为 true, 展示商品卡片
```

9. 记录点击信息（如果是普通商品）

```
用户点击单品返卡片中的商品
↓
clickProduct(product, productIndex)
↓
埋点上报
↓
跳转: openUrl(jumpUrl 或 商品详情页)
↓
页面变为 hidden
```

阶段4：用户回退和直通车插入

10. 页面回退

```
NewNormalItem 点击时:
→ mx.store.update(Store.currentGoodsItemId, itemId)
→ 记录到 clickedGoodsIdArray
```

11. 直通车回退逻辑

```
用户点击返回按钮
↓
页面变为 visible
↓
visibility.watch 触发
↓
updateFeedsVisible('visible')
```

12. 列表更新

```
检查: hitZtcAdBack === 'true'
↓
获取: currentGoodsItemId 从 store
↓
请求: queryMajorFeedsZtcBack({ itemId })
↓
插入: internalListRef.current.updateListInfo()
→ 在点击商品后面插入直通车商品
→ 重新计算 trueIndex
```

→ 重新计算 cruiseIndex

阶段5：问题发生点

13. 组件状态丢失（问题根源）

```
updateListInfo 执行
↓
列表数据更新
↓
renderCard 重新执行
↓
所有卡片重新渲染
```

关键数据流

```
情况A：组件重新挂载（key 变化或位置变化）
→ products 状态重置为 []
→ 但 requestedCardsRef 中已记录 cardId
→ requestSingleItemFeeds 检查到已请求，直接返回
→ products.length === 0
→ shouldRender = false
→ 组件不展示 ❌

情况B：组件只重新渲染（key 不变）
→ products 状态保留
→ 组件正常展示 ✅
```

问题总结

- 原因：组件重新挂载时，`products` 状态丢失，但 `requestedCardsRef` 已记录，不会再次请求。
- 影响：直通车回退后，单品返卡片不展示。
- 解决方向：在父组件缓存 `products` 数据，组件重新挂载时从缓存恢复。

需要我实现缓存方案吗？

缓存的方案时最简单最优的方案吗?

方案对比分析

方案1：缓存方案（之前提出的）

```
// 父组件
const cachedProductsRef = useRef(new Map<string, any[]>());

// 子组件挂载时从缓存恢复
useEffect(() => {
  const cached = cachedProductsRef.current.get(cardId);
  if (cached) {
    setProducts(cached);
  }
}, []);
```

优点：

- 实现简单
- 能解决问题

缺点：

- 需要额外内存存储
- 数据可能过期
- 需要手动管理缓存生命周期

方案2：状态提升方案（推荐）

```
// 父组件管理 products 状态
const [singleItemProducts, setSingleItemProducts] = useState<Map<string, any[]>>(new Map());

// 作为 props 传入子组件
<SingleItemChannelCard
  products={singleItemProducts.get(cardId) || []}
  onProductsLoaded={(cardId, products) => {
    setSingleItemProducts(prev => new Map(prev).set(cardId, products));
  }}
/>
```

优点：

- 符合 React 状态提升原则
- 状态在父组件，不会因子组件重新挂载而丢失
- 无需手动管理缓存
- 数据流更清晰

缺点：

- 需要修改组件接口（但改动不大）

方案3：检查是否真的需要重新挂载

如果 key 稳定，理论上不应重新挂载。可能是：

- `renderCard` 每次创建新函数
- 列表更新导致组件位置变化

但即使修复了重新挂载，状态提升仍是更稳健的做法。

方案4：使用 useMemo 持久化

```
const productsRef = useRef<any[]>([]);
// 但这样还是可能丢失
```

不推荐：ref 在重新挂载时也会重置。

最优方案：状态提升

原因：

1. 根本解决：状态在父组件，不受子组件挂载影响
2. 符合 React 实践：数据向上提升，单一数据源
3. 改动小：只需将 `products` 从子组件移到父组件
4. 无需缓存管理：直接用 state 管理

实现思路：

```
// 父组件
const [singleItemProductsMap, setSingleItemProductsMap] = useState(
  new Map<string, any[]>()
);

// 子组件请求成功后
onProductsLoaded(cardId, products) {
  setSingleItemProductsMap(prev => {
    const newMap = new Map(prev);
    newMap.set(cardId, products);
    return newMap;
  });
}

// 作为 props 传入
<SingleItemChannelCard
  products={singleItemProductsMap.get(cardId) || []}
  onProductsLoaded={handleProductsLoaded}
/>
```

这样即使组件重新挂载，`products` 也会从父组件的 state 中恢复，不会丢失。

需要我实现状态提升方案吗?

实现总结

1. 父组件（Feeds/index.tsx）

- 添加 `singleItemProductsMap` 状态，使用 `Map<string, any[]>` 存储每个卡片的商品数据
- 添加 `handleProductsLoaded` 回调，用于接收子组件加载完成的商品数据
- 将 `products` 和 `onProductsLoaded` 作为 props 传递给子组件

2. 子组件（SingleItemChannelCard/index.tsx）

- 移除内部的 `products` state
- 从 props 接收 `products` 数据
- 添加 `onProductsLoaded` 回调 prop
- 请求成功后调用 `onProductsLoaded` 通知父组件更新数据
- 渲染逻辑使用从 props 传入的 `products`

核心优势

1. 状态不丢失：即使组件重新挂载，`products` 数据在父组件中保留
2. 符合 React 实践：状态提升，单一数据源
3. 无需缓存管理：使用 state 管理，自动处理更新
4. 逻辑清晰：数据流向明确，父组件管理，子组件展示

工作流程

```
1. 子组件请求成功
   ↓
2. 调用 onProductsLoaded(cardId, products)
   ↓
3. 父组件更新 singleItemProductsMap
   ↓
4. 父组件重新渲染，传递新的 products 给子组件
   ↓
5. 子组件接收到新的 products，正常展示
   ↓
6. 即使组件重新挂载，products 也会从父组件的 state 中恢复
```

现在即使直通车回退导致组件重新挂载，商品数据也不会丢失，因为数据存储在父组件的 state 中。可以测试一下。

再确认逻辑，确认其他优化：

- 数据请求失败，不清空数据，只清楚标识，允许重试
- updateFeedsData事件：清空数据

问题总结：

- 核心问题：单品返渠道插卡作为主页Feeds依赖的子组件，数据只需要请求一次，但因为依赖数组中包含了首页曝光商品，直通车回退时
  - 曝光商品改变
  - 组件重新挂载（点击的商品在渠道卡片之前），导致状态丢失，接口重新请求
- 解决方式：将渠道卡片的Feeds请求的状态和数据提升到父组件中，保证当前会话不变。

3. 页面加载状态优化

因为首页需要商品去重，首页单品返渠道卡片Feeds的请求和主页Feeds存在时序，cursor一开始实现时，采用双状态：page1Loaded、page2Loaded来分别记录页面的加载状态，但综合来开，最后对状态的使用时机，单个状态即可满足要求，让AI帮忙优化。

```
if (page1Loaded && page2Loaded && !hasRequested &&
!loading) {
  requestSingleItemFeeds();
}
```

加载单品返Feeds判断

分析下page1Loaded, page2Loaded的设置和使用，看有没有其他实现方式？现在的实现有没有优化的空间

Thought for 2s

让我分析一下 `page1Loaded` 和 `page2Loaded` 的完整实现：

Explored 2 files 2 searches

当前实现分析

📊 完整流程图

```
父组件 (Feeds/index.tsx)
├─ useState: page1Loaded = false
├─ useState: page2Loaded = false
│
├─ SSR 场景: useEffect → setPage1Loaded(true)
│
├─ 请求完成: updatePageLoadedStatus(pageNo)
│   ├─ pageNo === 1 → setPage1Loaded(true)
│   └─ pageNo === 2 → setPage2Loaded(true)
│
└─ 传递给子组件 ↓

子组件 (SingleItemChannelCard/index.tsx)
├─ 接收 props: page1Loaded, page2Loaded
│
└─ useEffect 监听变化
    └─ if (page1Loaded && page2Loaded && !hasRequestedRef && !loading)
        └─ setTimeout(() => requestSingleItemFeeds(), 0) ⚠️
```

❗ 存在的问题

问题	描述	影响
1. 状态冗余	两个 boolean，实际只需要一个状态值	浪费状态
2. 多次渲染	每个状态变化都触发父组件重新渲染	性能损耗
3. 复杂的条件判断	page1Loaded && page2Loaded	代码复杂
4. 需要 setTimeout	用户添加了 setTimeout(0) 说明有时序问题	不稳定
5. Props 传递链路长	父组件 → 子组件，两个 props	维护成本高

📊 方案对比

方案	渲染次数	代码复杂度	可维护性	推荐度
当前实现	高（2个状态）	中	中	⭐⭐
方案1：单状态	中（1个状态）	低	高	⭐⭐⭐⭐
方案2：useRef + 触发状态	低（1次触发）	中	高	⭐⭐⭐⭐⭐
方案3：回调注册	最低（0状态传递）	低	最高	⭐⭐⭐⭐⭐

🎯 推荐实现

我推荐**方案1（单状态）**，原因是：

1. **改动最小**：只需修改状态定义和判断条件
2. **性能提升明显**：渲染次数减半
3. **代码更清晰**：`loadedPageCount >= 2` 比 `page1Loaded && page2Loaded` 更直观
4. **不需要 setTimeout**：时序问题自然解决

需要我帮您实现方案1吗？

不不不，不要这样改，请将page1Loaded和page2Loaded两个状态合并成一个firstTwoPagesLoaded状态

Thought for 2s

好的！我理解了，将两个布尔状态合并成一个 `firstTwoPagesLoaded` 布尔状态。让我重新实现：

index.tsx +6 -3

```
const [loadedPageCount, setLoadedPageCount] = useState(0);
// 前两页加载状态（用于控制单品返渠道卡片的请求时机）
const [firstTwoPagesLoaded, setFirstTwoPagesLoaded] = useState(false);
// 单品返渠道卡片已展示的商品ID（用于第三页及以后的Feeds去重）
```

2 个隐藏的行

index.tsx +5 -1

```
  loadedPagesRef.current.add(pageNo);
  // 当第1页和第2页都加载完成时，更新状态
  if (loadedPagesRef.current.has(1) && loadedPagesRef.current.has(2)) {
    setFirstTwoPagesLoaded(true);
  }
```

紧急撤回...

给我急得...

都这么说了，蒜了蒜了

4. 单品渠道卡片区分单商品和多商品，组件优化

AI在实现时，是根据不同的渠道卡片不同的biztype，使用不同的css样式，来完成不同卡片样式的渲染，但服务端的经验告诉我们，应该抽出可复用的组件。让cursor抽取组件相同的样式文件、公共参数等，使得结构更加清晰。

STEP 5: 视觉验收、埋点

验收过程，测试阶段，通过截图或者让视觉体验，发现问题，根据要求修改。

埋点，让PD梳理好埋点相关信息，让AI执行，去埋点平台验证。

04 总结

AI coding充满想象力，也不断拓展着技术的边界。从这个具体的业务需求实践去看，虽然遇到了各种各样的问题，但借助AI去梳理逻辑、定位问题，在不需要完备的知识体系背景下，也能最终解决问题，达到预期效果，但还是会走不少弯路。所以全栈交付的过程中，还是建议去多了解技术栈的基础知识，再搭配AI工具，会更加的游刃有余。

还有一个AI coding问题排查的心得：描述问题，让AI去增加日志埋点，然后直接把控制台的输出丢给他。

1. **接口解析日志**：
   - `[querySingleItemCashBackFeeds] 方式1/2/3解析成功:` - 看哪种方式解析成功
   - `[querySingleItemCashBackFeeds] 无法解析数据` - 如果无法解析，会显示返回结构
   - `[querySingleItemCashBackFeeds] 最终返回数据:` - 最终返回的数据
2. **组件接收数据日志**：
   - `[SingleItemChannelCard] 接口返回:` - 组件接收到的接口数据
   - `[SingleItemChannelCard] 处理后的商品数据:` - 处理后的商品数据
   - `[FeedsList] 接收商品数据:` - 父组件接收到的数据
   - `[FeedsList] 更新后的Map:` - 更新后的 Map 状态
3. **渲染检查日志**：
   - `[SingleItemChannelCard] 渲染检查:` - 渲染条件的状态

请把控制台的完整日志输出发给我，我会根据日志定位问题。

05 团队介绍

本文作者凌由，来自淘天集团–用户互动团队。团队支撑数亿级用户的互动业务（淘金币、芭芭农场、红包签到）的产品能力建设及演进，支持双十一、春节大促互动玩法的架构和开发，保障数亿权益的精准发放，通过多种互动形式提升用户规模和订单增长。我们正承担着捍卫电商主板块增长的重要使命，是阿里核心电商战场的参与者，用持续的技术创新来驱动阿里电商引擎的稳步前行。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法

大淘宝技术

大淘宝技术官方账号

967篇原创内容

公众号

阅读 1.3万　修改于2026年2月6日

留言 3

写留言

Frank 江苏 2月10日　1

这是一篇没有吹牛的纪实文章，用过的都懂

林新建 浙江 2月9日　1

真实感拉满，浓浓的给ai擦屁股的感觉

无忌 浙江 2月6日

很专业、很前沿。

深蓝汽车销售号　广告

深蓝L06，至低10.1万元起，限时享至高24792元综合优惠！

进入小程序